



Chitkara

PAPER OF FUNDAMENTALS OF C PROGRAMMING FA 4 - CSE(AI) 2025 BATCH - E EXAM

Q1. Convert upper case to lower case

Write a program that reads a string containing **uppercase letters** and converts all characters in the string to **lowercase**.

Input Format

The first line contains a string.

Output Format

Print the string after conversion to lower case

Constraints

$1 \leq \text{length of string} \leq 1000$
String contains only uppercase alphabets (A-Z)
No special characters or digits

Sample Input

CODEQUOTIENT

Sample Output

codequotient

Explanation

The input string is:

CODEQUOTIENT

After converting each character to its **lowercase equivalent**, the resulting string becomes:

codequotient

ANSWER:

Test case 1 :

Input:
CODEQUOTIENT
Output:
codequotient

Test case 2 :

Input:
HELLO
Output:
hello

Test case 3 :

Input:
PROGRAMMING
Output:
programming

Test case 4 :

Input:
GOOD
Output:
good

Test case 5 :

Input:
LANGUAGE
Output:
language

Test case 6 :

Input:
XYZ
Output:
xyz

Q2. Perfect String

Given a string that contains only 4 type of letters i.e. a , b , x , and y . The string is said to be perfect if it satisfies the following conditions:

1. count of 'a' + count of 'b' in the given string must be odd.
2. count of 'x' + count of 'y' in the given string must be even.

Note: It is possible that in some string all the 4 type of letters are not present. for example, "aab" is also a perfect string because $a_count = 2$, $b_count = 1$, $x_count = 0$, $y_count = 0$ which satisfies the above conditions.

Complete the function ***isPerfect()*** in the editor. The function receives a string to test as parameter and must return true or false based on the fact that, whether the given string is perfect or not.

Input Format:

The first line contains an integer T , denoting the number of test cases.
Each test cases contains one line, which contains a string.

Output Format:

For each test case, print YES if the string is perfect, else print NO.

Constraints

$1 \leq T \leq 100$
 $1 \leq \text{length of each string} \leq 10^5$

Strings contain only the letters 'a', 'b', 'x', 'y'.

Sample Input

```
4
xaxbyyb
aabbxxyy
abaabxy
yyxaby
```

Sample Output

```
YES
NO
YES
NO
```

Explanation:

Only first and third string is perfect.

ANSWER:

Test case 1 :

Input:

```
4
xaxbyyb
aabbxxyy
abaabxy
yyxaby
```

Output:

```
YES
NO
YES
NO
```

Test case 2 :

Input:

```
4
yaayyybbbb
aaabbbaxxxx
bbbbxxaayxyxyxy
aaaaaaaaaaaaa
```

Output:

```
YES
YES
NO
YES
```

Test case 3 :

Input:

```
4
aaaaxxyybbbyxxaabbyyxx
xybaxybabxyabxayxyabbxyaxb
bbbbbbbbbbxxxxxxxxxxxxxxxxxyyyyyyaaaaaaaaaxbx
yyyaabbxxxxbayxbayxxbbaxybaxybayxbayxbyybxa
```

Output:

```
NO
```

YES
NO
NO

Test case 4 :

Input:

4
bbbbbbbbbbbb
ayyyaaaabbxxxxbayxbayxxbbaxybaxybayxbayxbyybxa
xxxxxxxxxxxxxxxxxxxxxx
yyyyyyyyyyyyyyyyyyyyyy

Output:

YES
NO
NO
NO

Test case 5 :

Input:

4
xybaxybabxyabybxaxbaxayxyabbxyaxb
byxaybxxbybxybaxbyabyxaxbyaxbyy
aaaaaaaaaaaaaaaa
yyyyyyyyyyyyyyyyyyyyyy

Output:

NO
YES
YES
NO

Test case 6 :

Input:

3
aaaaxxyxyaybbbyxxaabbyyxx
bbbbxbbbbxxxxaabbyxxxxxxxxxyyyyyyaaaabaaaaxbx
byxaybxxbybxybaxbyaxbyaxby

Output:

NO
NO
YES

Q3. Evaluate Recurrence relation

Complete the given function to implement a mathematical recurrence relation $T(k)$.

A recurrence relation $T(n)$ is defined for all $n \geq 0$ as:

- $T(0) = 1$
- $T(n) = T(n-1) + 2 \times n$, for $n \geq 1$

You are given an integer k . Your task is to **compute and return the value of $T(k)$** .

The solution **must use recursion**.

Input Format

First line contains an integer n , number of test cases.
Next n lines each contain one integer k .

Output Format

Function should return the integer result of $T(k)$ for input k .

Constraints

Number of test cases (n): $1 \leq n \leq 10000$
Value of k : $0 \leq k \leq 1000$

Sample Input

```
2
1
2
```

Sample output

```
3
7
```

Explanation

For $k=1$:
 $T(1) = T(0) + 2*1 = T(0) + 2$
Since $T(0) = 1$, result is $1 + 2 = 3$.

For $k=2$
 $T(2) = T(1) + 2*2 = T(1) + 4$
 $T(1) = T(0) + 2*1 = T(0) + 2$
Since $T(0) = 1$, result is $(1 + 2) + 4 = 7$.

ANSWER:

Test case 1 :

Input:

```
2
1
2
```

Output:

```
3
7
```

Test case 2 :

Input:

```
2
400
450
```

Output:

```
160401
202951
```

Test case 3 :

Input:

```
5
```

0
1
2
3
5

Output:

1
3
7
13
31

Test case 4 :

Input:

5
4
6
7
10
15

Output:

21
43
57
111
241

Test case 5 :

Input:

2
0
3

Output:

1
13

Test case 6 :

Input:

3
6
1
2

Output:

43
3
7

Q4. power(base, exp)

Write a recursive function **power** that accepts two integers representing a base and an non-negative exponent and returns the base raised to that non-negative exponent. For example, the call of power(3, 4) should return 81 . If the exponent passed is negative, throw an exception or print error message.

Note: If the exponent is negative, the program must throw an exception (for languages like **C++/Java/Python**) or print this exact error message: Negative exponent not allowed (for C language).

Do not use loops or auxiliary data structures; Solve the problem **recursively**.

Also do not use the provided in-built library power function in your solution.

Input Format

The first line contains a single integer T, number of test cases.

Next T lines contains two space-separated integers, base and exp for each test case.

Output Format

For each test case, the result of power(base, exp) will be returned by function and printed from main or error message will be printed from function itself.

Each result is printed on a new line.

If the exponent is negative, you only have to print an error message "Negative exponent not allowed" (for C language) or throw an exception (depending on languages like C++/Java/Python).

Constraints

$1 \leq T \leq 10^5$

$-10^5 \leq \text{base} \leq 10^5$

$0 \leq \text{exp} \leq 10^5$

Sample Input

```
3
3 4
2 5
5 0
```

Sample Output

```
81
32
1
```

Explanation

For power(3,4): = 81

power(2,5) = 32

power(5,0) = 1

$3^4 = 3 * 3 * 3 * 3 = 81.$

For power(5, 0):

By mathematics definition, any non-zero base raised to the power of 0 results in 1. So output is 1.

ANSWER:

Test case 1 :

Input:

```
6
3 4
5 4
2 10
-5 3
7 1
0 0
```

Output:

```
81
625
1024
-125
```

7

1

Test case 2 :

Input:

3

3 4

2 5

5 0

Output:

81

32

1

Test case 3 :

Input:

1

3 3

Output:

27

Test case 4 :

Input:

1

3 -3

Output:

Negative exponent not allowed

Test case 5 :

Input:

3

1 1

3 -5

6 -11

Output:

1

Negative exponent not allowed

Negative exponent not allowed

Test case 6 :

Input:

4

4 2

55 3

11 2

11 -11

Output:

16

166375

121

Negative exponent not allowed

Q5. Sum of All Digits (Recursion)

Write a recursive function `sumOfDigits()` that accepts an integer as a parameter and returns the sum of its digits. The function must handle both positive and negative integers according to the specific rules below.

Rules:

If n is **positive**, return the sum of its digits.

Example: $1729 \rightarrow 1 + 7 + 2 + 9 = 19$

If n is **negative**, return the negative of the digit sum of its absolute value.

Example: $-1729 \rightarrow -(1 + 7 + 2 + 9) = -19$

Note:

- Use recursion only
- Do not use loops
- Do not use string conversion

Input Format:

The first line contains an integer T , the number of test cases.
Each of the next T lines contains a single integer n .

Output Format:

For each test case, print the sum of the digits on a new line.

Constraints:

$1 \leq T \leq 100$
 $-10^9 \leq n \leq 10^9$

Sample Input:

```
3
1729
-1729
502
```

Sample Output

```
19
-19
7
```

Explanation

For a positive number like 1729, the function should return $1 + 7 + 2 + 9 = 19$.

For a negative number like -1729, the function should return the negative of the sum of its absolute digits, which is -19.

For a positive number like 502, the function should return $5 + 0 + 2 = 7$.

ANSWER:

Test case 1 :

Input:

```
3
1729
-1729
```

502

Output:

19

-19

7

Test case 2 :

Input:

3

10

25

123

Output:

1

7

6

Test case 3 :

Input:

2

-45

89

Output:

-9

17

Test case 4 :

Input:

4

0

9

-9

10

Output:

0

9

-9

1

Test case 5 :

Input:

3

1001

9000

-10002

Output:

2

9

-3

Test case 6 :

Input:

2

99999

123456

Output:

45

21

Q6. Reverse a string using recursion

Complete the function **reverseString** that takes a string as input and returns the string reversed.

Input Format

A single line containing the string `s`.

Output Format

Print the reversed string.

Constraints

$1 \leq \text{length of } s \leq 10^5$

The string may contain letters (uppercase/lowercase), digits, spaces, and special characters.

Example 1

Input: "hello"

Output: "olleh"

Explanation

The string "hello" has 5 characters: h, e, l, l, o.

Reversing swaps the first and last characters, then the second and second-last, keeping the middle character the same.

Resulting string: "olleh".

Example 2

Input: "A man, a plan, a canal: Panama"

Output: "amanaP :lanac a ,nalp a ,nam A"

Explanation

Every character, including spaces and punctuation, is reversed.

The order of letters, spaces, commas, and colons is mirrored.

ANSWER:

Test case 1 :

Input:

codequotient

Output:

tneitouqedoc

Test case 2 :

Input:

programming

Output:

gnimmargorp

Test case 3 :

Input:
"hello"
Output:
"olleh"

Test case 4 :

Input:
"A man, a plan, a canal: Panama"
Output:
"amanaP :lanac a ,nalp a ,nam A"

Test case 5 :

Input:
"MixedCASEstring"
Output:
"gnirtsESACdexiM"

Test case 6 :

Input:
" Leading and trailing spaces "
Output:
" secaps gniliart dna gnidaeL "

Test case 7 :

Input:
"12345"
Output:
"54321"

Q7. Reverse an array using pointers

Write a function `void reverseArray(int *a, int size)` that reverses the elements of an array using **pointers**.

The function receives:

- a pointer to the first element of the array
- an integer representing the size of the array

The function must **reverse the elements inside the array itself**.

Note: Do **not print anything inside the function**, as printing is handled in the `main` function.

Input Format

The first line contains an integer `n`, representing the number of elements in the array.
The second line contains `n` space-separated integers.

Output Format

Print the array elements after reversal, separated by a space.

Constraints

$1 \leq n \leq 1000$

Sample Input

5
1 2 3 4 5

Sample Output

5 4 3 2 1

Explanation

Input array:

1 2 3 4 5

After reversing the array using pointers, the order of elements becomes:

5 4 3 2 1

ANSWER:

Test case 1 :

Input:

5
1 2 3 4 5

Output:

5 4 3 2 1

Test case 2 :

Input:

7
1 2 3 4 5 6 7

Output:

7 6 5 4 3 2 1

Test case 3 :

Input:

10
2 4 6 8 10 1 3 5 7 9

Output:

9 7 5 3 1 10 8 6 4 2

Test case 4 :

Input:

9
10 20 30 40 50 60 70 80 90

Output:

90 80 70 60 50 40 30 20 10

Test case 5 :

Input:

2
10 20

Output:

20 10

Test case 6 :

Input:

6

34 5 21 7 89 87

Output:

87 89 7 21 5 34

Q8. Return multiple values from array

In C, parameters can be passed to a function either **by value** or **by reference**. When parameters are passed by reference (using pointers), any changes made inside the function are reflected in the calling function.

Using this concept, you can return **multiple values from a function** by passing variables as pointer parameters.

Your task is to Complete the function:

```
void multiReturnArray(int *a, int size, int *min, int *max, int *sum)
```

The function should:

- Take an array a of size size
- Compute:
 - Minimum element of the array
 - Maximum element of the array
 - Sum of all elements
- Store the results in the variables pointed to by:
 - min
 - max
 - sum

Note: Do not print anything inside the function, as printing is handled in the main function.

Input Format

The first line: An integer representing the total number of elements in the array.

The second line: A sequence of space-separated integers representing the elements of the array.

Output Format

Three integers separated by spaces:

1. The minimum value found in the array.
2. The maximum value found in the array.
3. The sum of all elements in the array.

Constraints

Array Size (N): $1 \leq N \leq 10^5$

Value Range: $-10^4 \leq \text{arr}[i] \leq 10^4$

Sample Input

5

1 3 5 7 9

Sample Output

1 9 25

Explanation

Input array is {1, 3, 5, 7, 9}.

Minimum: The smallest value in the set is 1.

Maximum: The largest value in the set is 9.

Sum: $1 + 3 + 5 + 7 + 9 = 25$.

ANSWER:

Test case 1 :

Input:

5

1 3 5 7 9

Output:

1 9 25

Test case 2 :

Input:

7

1 2 3 4 5 6 7

Output:

1 7 28

Test case 3 :

Input:

10

2 4 6 8 10 1 3 5 7 9

Output:

1 10 55

Test case 4 :

Input:

9

10 20 30 40 50 60 70 80 90

Output:

10 90 450

Test case 5 :

Input:

6

2 4 5 7 9 11

Output:

2 11 38

Test case 6 :

Input:

8

2 3 4 5 6 7 8 9

Output:

2 9 44

Q9. Remove Duplicate Elements

Rajesh was working with students' records. He has given a list of N element contains the roll numbers of the students, but there may be some duplicate entries. Write a function **void removeduplicate (int* arr, int* size);**and remove duplicate entries while **preserving the original order**.

Note: Do not print anything inside the function, as printing is handled in the main function.

Input Format:

First line: integer N (number of elements)
Second line: N space-separated integers

Output Format:

Print elements after removing duplicates (in same order)

Constraints

$1 \leq N \leq 10^5$
 $1 \leq \text{element} \leq 10^9$

Sample Input:

```
10
3 8 4 3 14 5 8 3 5 11
```

Sample Output:

```
3 8 4 14 5 11
```

Explanation :

Given array is: (3 8 4 3 14 5 8 3 5 11)
and size = 10

In the above array, **3 has two more occurrences after position 0**, so we delete those duplicates. After shifting the elements, the size becomes **8**.

Now the remaining array is:(3 8 4 14 5 8 5 11)
and size = 8

Next, **8 appears one more time after index 1**, so we delete that duplicate. The size becomes **7**.

Now the remaining array is: (3 8 4 14 5 5 11)
and size = 7

Next, **5 occurs more than once**, so we delete one occurrence of 5. Now the size becomes **6**.

Finally, the remaining array is: (3 8 4 14 5 11)
and size = 6

ANSWER:

Test case 1 :

Input:
10
3 8 4 3 14 5 8 3 5 11
Output:
3 8 4 14 5 11

Test case 2 :

Input:
6
1 2 3 4 5 5
Output:
1 2 3 4 5

Test case 3 :

Input:
15
1 1 1 2 2 3 4 5 6 1 3 8 4 5 9
Output:
1 2 3 4 5 6 8 9

Test case 4 :

Input:
12
5 8 6 3 5 6 3 5 8 5 2 7
Output:
5 8 6 3 2 7

Test case 5 :

Input:
4
3 3 3 3
Output:
3

Test case 6 :

Input:
8
5 4 7 9 3 4 6 9
Output:
5 4 7 9 3 6

Q10. DECAYING SERIES

Develop a function **void decay(int *iNum)** in C to generate a **decaying series** based on a given integer T.

The series continues until it reaches **1**.

The next number in the series is determined by the following rules:

- If the current number is **even** → divide it by 2
- If the current number is **odd** → multiply it by 3 and add 1

The generated values (excluding the initial input T) form the required series.

Input Format

A single line contains an integer T, who's access is given in decay() function via pointer.

Output Format

For given number, print the resulting series (excluding the initial input T) separated by spaces on a new single line.

Constraints

Intital number (T): $1 \leq n \leq 10^9$

Sample Input 1

17

Sample Output 1

52 26 13 40 20 10 5 16 8 4 2 1

Explanation

Step 1: Given 17 is an odd no.
Step 2: So $(17*3)+1 = (51+1) = 52$ will be printed on screen.
Step 3: Now 52 is an even number, so $52/2 = 26$ will be printed on screen.
Step 4: Since 26 is an even number, so $26/2 = 13$ will be printed on screen.
Step 5: Now 13 is odd, so $(13*3)+1 = 40$ will be printed.
Step 6: Since 40 is even, so $40/2 = 20$ will be printed on screen.
Step 7: Now 20 is even, so $20/2$ i.e. 10 will be printed.
Step 8: Now $10/2 = 5$ will be printed.
Step 9: As 5 is odd, $(5*3)+1 = 16$ will be printed.
Step 10: Now 16 is even, so $16/2 = 8$ will be printed.
Step 11: Now $8/2 = 4$ will be printed.
Step 12: Now $4/2 = 2$ will be printed.
Step 13: Now $2/2 = 1$ will be printed and as series goes upto 1 so we stop here.

Sample Input 2

40

Sample Output 2

20 10 5 16 8 4 2 1

Explanation

Step 1: Since 40 is even, so $40/2$ i.e 20 will be printed on screen.
Step 2: Now 20 is even, so $20/2$ i.e. 10 will be printed.
Step 3: Now $10/2 = 5$ will be printed.
Step 4: As 5 is odd, $(5*3)+1 = 16$ will be printed.
Step 5: Now 16 is even, so $16/2 = 8$ will be printed.
Step 6: Now $8/2 = 4$ will be printed.
Step 7: Now $4/2 = 2$ will be printed.
Step 8: Now $2/2 = 1$ will be printed and as series goes upto 1 so we stop here.

ANSWER:

Test case 1 :

Input:

17

Output:

52 26 13 40 20 10 5 16 8 4 2 1

Test case 2 :

Input:

46

Output:

23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1

Test case 3 :

Input:

12

Output:

6 3 10 5 16 8 4 2 1

Test case 4 :

Input:

5

Output:

16 8 4 2 1

Test case 5 :

Input:

2

Output:

1

Test case 6 :

Input:

13

Output:

40 20 10 5 16 8 4 2 1

Q11. Delete an element from a specified index

Sachin wants to delete an element from a particular index in an array of N elements. Help him in making a coding solution for the same. Given deletion index will follow 0-based indexing.

Note: Printing the output is not required, as it is already handled in the main function.

Input Format

First line contains n: The number of elements in the array.

Second line contains Array Elements: n space-separated integers.

Third line contains Index: The position (0-indexed based) of the element to be deleted.

All of these are passed to the function deletion() and accessible in it.

Output Format

Function deletion() have to update the array by performing deletion at given index.

Result will be automatically printed by given main() function.

Constraints

Array Size (n): $1 \leq n \leq 1000$

Deletion Index (index): $0 \leq \text{index} \leq n-1$

Array element's value (v): $-100000 \leq v \leq 100000$

Sample Input

6

23 25 26 24 56 20

3

Sample Output

23 25 26 56 20

Explanation

For given Sample Input:

Array Size = 6, Array = {23, 25, 26, 24, 56, 20}, Deletion Index = 3

So first we identify the element at Index 3: 24.

Shift elements after index 3 :-

Move the element at index 4 (56) to index 3.

Move the element at index 5 (20) to index 4.

After deletion, the array becomes: {23, 25, 26, 56, 20, 20}.

Now provided main() function automatically prints the first N-1 elements: 23 25 26 56 20

ANSWER:

Test case 1 :

Input:

6

3 25 26 24 56 20

3

Output:

3 25 26 56 20

Test case 2 :

Input:

10

4 25 26 24 56 89 2 63 8 4

1

Output:

4 26 24 56 89 2 63 8 4

Test case 3 :

Input:

10

2 5 6 2 6 8 2 6 8 4

1

Output:

2 6 2 6 8 2 6 8 4

Test case 4 :

Input:

10

3 5 26 24 5 89 2 3 8 4

2

Output:

3 5 24 5 89 2 3 8 4

Test case 5 :

Input:

10

3 5 26 24 5 89 2 3 8 4

1

Output:

3 26 24 5 89 2 3 8 4

Test case 6 :

Input:
5
23 25 26 24 56
3
Output:
23 25 26 56

Q12. Average of odd and even numbers in an array

Employees of a company are going for a professional training. There are total 6 boys and girls in company who are going for training. You are required to enter the salary of employees and find out the average of salary of boys and girls using one-dimensional array.

Note:

Boy's salary: Any Odd integer value

Girl's salary: Any Even integer value

Printing the output is not required, as it is already handled in the main function.

Input Format

A single line contains 6 space-separated integers representing employee salaries. These value's array is passed to function avgOddEven() and accessible via 'arr'.

Output Format

Your function should update calculated averages for both boys (odd) and girls (even) in respective pointers: *avgOdd and *avgEven. Printing the output is not required, as it is already handled in the main() function.

Constraints

Number of salaries : Fixed at 6
Salary Value: $1 \leq \text{salary} \leq 10^6$

Sample Input 1

15 10 15 20 25 30

Sample Output 1

18.33
20.00

Sample Input 2

12 10 13 14 11 9

Sample Output 2

11.00
12.00

Explanation

For Sample Input-1 : 15 10 15 20 25 30
Identify Boys (Odd) Salaries: {15, 15, 25}
Sum: $15 + 15 + 25 = 55$ Count: 3.
Average: $55 / 3 = 18.33$ (Printed)

Identify Girls (Even) Salaries: {10, 20, 30}

Sum: 10 + 20 + 30 = 60 Count: 3

Average: 60 / 3 = 20.00 (Printed)

ANSWER:

Test case 1 :

Input:

15 10 15 20 25 30

Output:

18.33

20.00

Test case 2 :

Input:

12 10 13 14 11 9

Output:

11.00

12.00

Test case 3 :

Input:

12 11 11 14 9 2

Output:

10.33

9.33

Test case 4 :

Input:

14 15 8 7 6 17

Output:

13.00

9.33

Test case 5 :

Input:

11 22 53 69 84 48

Output:

44.33

51.33

Test case 6 :

Input:

14 63 33 21 15 16

Output:

33.00

15.00

Test case 7 :

Input:

32 99 15 81 77 54

Output:

68.00

43.00

